



AE10. Avances de proyecto

Brandon Chavarría Ángeles

Kenneth Morán Gómez

Emmanuel Islas Lozada

José Enrique Sánchez Romero

Universidad Politécnica de Pachuca

Mantenimiento de Software

Alicia Ortiz Montes

17 de Agosto de 2026



Índice

Introducción	5
Antecedentes del Estudio	5
Alcance de este Reporte	5
Metodología	6
Resultados	7
Contraste entre Planeación y Avance Real	7
Gráfica de Quemado (Burndown Chart)	8
Discusión	9



Índice de figuras

1.	Tablero de GitHub Projects del Proyecto Yoliztli	6
2.	Gráfica de Quemado — Estado Real al 17 de Agosto de 2026	8



Índice de cuadros

1. Avance real contrastado con la planeación por sprint 7

Introducción

El presente documento reporta el avance real del proyecto de mantenimiento de software del sistema Yoliztli, contrastado contra la planeación SCRUM establecida por el equipo. El objetivo de este entregable (AE10) es documentar, con evidencia verificable, el estado del backlog al 17 de agosto de 2026, la herramienta de seguimiento utilizada (GitHub Projects) y la gráfica de quemado (*burndown chart*) correspondiente al sprint en curso.

Antecedentes del Estudio

La planeación original del equipo estableció un backlog total de 20 puntos de historia (SP), distribuidos en dos sprints de dos semanas cada uno: Sprint 1 (Infraestructura y Datos, 13 SP) y Sprint 2 (Lógica y Calidad, 7 SP). Dicha planeación derivó del diagnóstico inicial documentado mediante SonarQube y de las pruebas funcionales de caja negra aplicadas al sistema.

Alcance de este Reporte

Este reporte cubre únicamente el corte de avance al cierre del Sprint 1, sin proyectar resultados de Sprint 2, que aún no ha iniciado formalmente.

Metodología

El seguimiento del proyecto se realizó mediante **GitHub Projects**, vinculado directamente al repositorio del código fuente. Se configuraron los siguientes elementos:

- Un tablero (*board*) con un *issue* por cada Solicitud de Modificación (MR-001 a MR-005), replicando la tabla de PR/MR de la planeación original.
- Campos personalizados de **Prioridad** (Crítica, Alta, Media, Baja) y **Sprint** (Sprint 1, Sprint 2), para agrupar y filtrar el backlog.
- Un flujo de trabajo (*workflow*) que vincula cada *Pull Request* (PR) a su *issue* correspondiente mediante la palabra clave `Closes #N`, de forma que el cierre del *issue* solo ocurre cuando el código respectivo ha sido fusionado a la rama principal.

Esta metodología distingue explícitamente dos niveles de avance: (1) *trabajo codificado*, es decir, funcionalidad ya programada pero pendiente de fusión, y (2) *trabajo verificado*, es decir, funcionalidad respaldada por un *issue* cerrado automáticamente vía PR fusionado. Esta distinción es la base de la gráfica de quemado presentada en la sección de Resultados.

Figura 1
Tablero de GitHub Projects del Proyecto Yoliztli

Title	Assignees	Status	Linked pull requests	Sub-issues progress	Prioridad	Sprint
1 MR-001: Reconstitución Frontend #1		Todo			Crítica	Sprint 1
2 MR-002: Remoción de Bugs #2		Todo			Alta	Sprint 2
3 MR-003: Deuda Técnica y Linting #3		Todo			Baja	Sprint 1
4 MR-004: Filtros Lógicos CP007 #4		Todo			Media	Sprint 2
5 MR-005: Persistencia CP008 #5		Todo			Alta	Sprint 1

Nota. Vista de tabla del Project mostrando los cinco *issues* (MR-001 a MR-005) con sus campos de Prioridad y Sprint asignados. Captura propia, 17 de agosto de 2026.

Resultados

Contraste entre Planeación y Avance Real

La Tabla 1 contrasta el backlog planeado por sprint contra el estado real observado en el tablero de GitHub Projects al momento del corte.

Tabla 1
Avance real contrastado con la planeación por sprint

Sprint	SP Planeados	SP Codificados	Estado en GitHub
Sprint 1 (Infraestructura y Datos)	13	13	PR abiertos, sin fusionar
Sprint 2 (Lógica y Calidad)	7	0	No iniciado

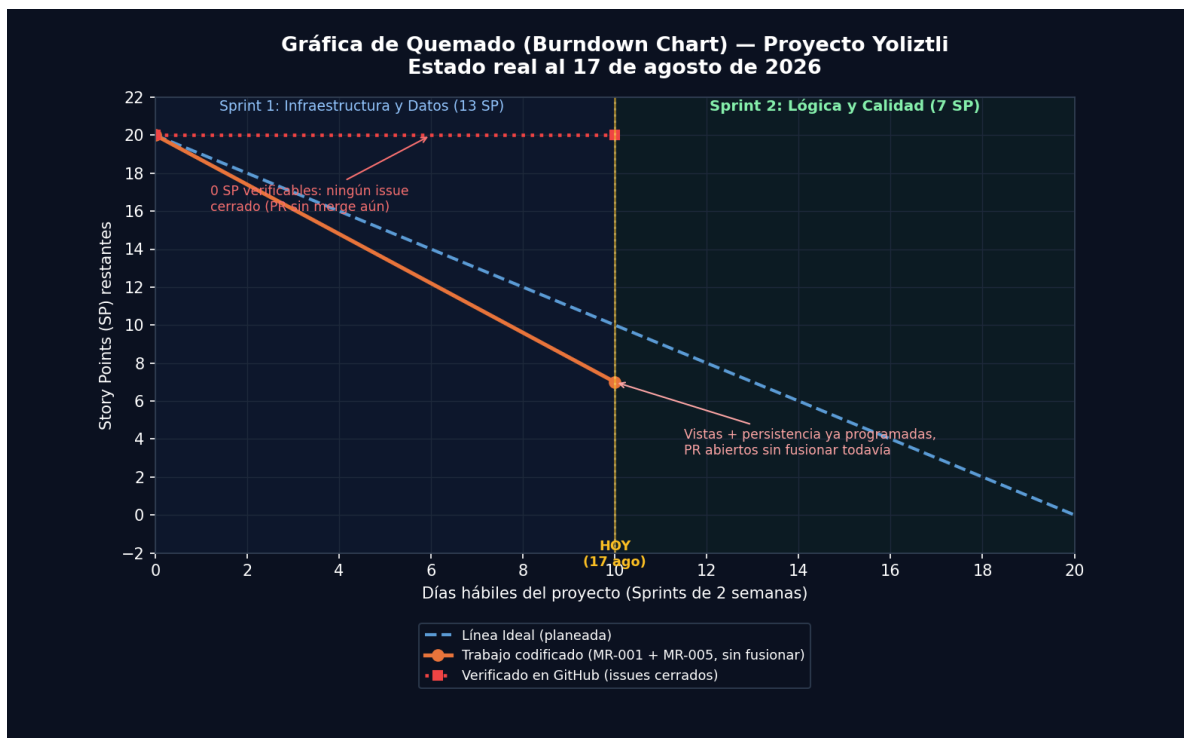
Nota. SP = Story Points. “Codificados” refiere a funcionalidad ya programada (vistas React reconstituidas y persistencia del progreso infantil); ninguna ha sido fusionada a `main` al corte de este reporte, por lo que el conteo verificable en GitHub es de 0 *issues* cerrados. Elaboración propia.

El equipo completó la programación del 100 % del alcance de Sprint 1 (MR-001: reconstitución del frontend; MR-005: persistencia del progreso infantil), lo cual coloca al equipo ligeramente adelantado respecto al ritmo ideal de una historia por día. Sin embargo, ninguno de los *Pull Requests* correspondientes ha sido fusionado a la rama principal al momento de este corte, por lo que el avance *verificable* mediante *issues* cerrados es de 0 SP.

Gráfica de Quemado (Burndown Chart)

La Figura 2 presenta la gráfica de quemado real del proyecto, distinguiendo la línea ideal planeada, el avance de trabajo codificado y el avance verificado formalmente en GitHub.

Figura 2
Gráfica de Quemado — Estado Real al 17 de Agosto de 2026



Nota. La línea punteada azul representa el ritmo ideal (20 SP en 20 días hábiles). La línea naranja representa el trabajo ya codificado (13 SP). La línea roja punteada representa el avance verificado mediante *issues* cerrados en GitHub (0 SP), dado que los *Pull Requests* de MR-001 y MR-005 permanecen abiertos. Elaboración propia.



Discusión

El contraste entre la línea de trabajo codificado y la línea de trabajo verificado evidencia una brecha de proceso más que una brecha de desarrollo: el equipo cumplió el alcance funcional de Sprint 1 dentro del tiempo planeado, pero el flujo formal de cierre de *issues* vía *Pull Request* fusionado aún no se ha completado. Esta brecha es de bajo riesgo y de resolución inmediata: basta con fusionar los PR abiertos de MR-001 y MR-005 incluyendo la palabra clave `Closes` en su descripción para que la línea verificada coincida con la línea de trabajo codificado.

Como siguiente paso, el equipo fusionará dichos *Pull Requests* antes de iniciar formalmente el Sprint 2 (Lógica y Calidad: MR-002, MR-003 y MR-004), de manera que cada sprint quede cerrado con evidencia trazable de *commits*, PR e *issues* antes de avanzar al siguiente.